

Cours Java

Pr. Allae Erraissi

Email : erraissi.a@ucd.ac.ma



Chapitre 9

La Généricité en Java

Introduction

- ❑ La généricité est une fonctionnalité essentielle de Java introduite avec Java 5. Elle permet de créer des classes, interfaces et méthodes capables de fonctionner avec différents types de données tout en garantissant une sécurité des types à la compilation.
- ❑ En d'autres termes, la généricité permet d'écrire un code réutilisable et flexible sans compromettre la fiabilité.
- Un exemple typique est l'utilisation des collections (comme ArrayList, HashMap) qui utilisent la généricité pour manipuler des éléments de différents types tout en évitant des erreurs fréquentes, comme les problèmes de conversion de types.

Introduction

❑ Exemple : Sans généricité

```
import java.util.ArrayList;

public class SansGenerique {
    public static void main(String[] args) {
        ArrayList liste = new ArrayList();
        liste.add(42); // Ajout d'un entier
        liste.add("Bonjour"); // Ajout d'une chaîne

        // Erreur potentielle de type : besoin d'un cast explicite
        String texte = (String) liste.get(0); // Peut causer une ClassCastException
        System.out.println(texte);
    }
}
```

Introduction

❑ Exemple : Avec généricité

```
import java.util.ArrayList;

public class AvecGenerique {
    public static void main(String[] args) {
        ArrayList<String> liste = new ArrayList<>();
        liste.add("Bonjour");
        // liste.add(42); // Erreur de compilation

        String texte = liste.get(0); // Pas besoin de cast explicite
        System.out.println(texte);
    }
}
```

Création d'une Classe Générique

- ❑ Une classe générique est définie à l'aide de types paramétrés.
- ❑ **Syntaxe de base :**

```
public class MaClasseGenerique<T> {  
    private T element;  
  
    public MaClasseGenerique(T element) {  
        this.element = element;  
    }  
  
    public T getElement() {  
        return element;  
    }  
  
    public void setElement(T element) {  
        this.element = element;  
    }  
}
```

Création d'une Classe Générique

❑ Exemple : Classe Générique pour Différents Types

```
public class TestGenerique {  
    public static void main(String[] args) {  
        MaClasseGenerique<Integer> instanceEntier = new MaClasseGenerique<>(100);  
        System.out.println("Valeur : " + instanceEntier.getElement()); // Affiche 100  
  
        MaClasseGenerique<String> instanceString = new MaClasseGenerique<>("Salut");  
        System.out.println("Valeur : " + instanceString.getElement()); // Affiche Salut  
    }  
}
```

Création de méthodes Génériques

- ❑ Les méthodes génériques permettent de définir des méthodes qui fonctionnent avec des types spécifiés au moment de l'appel.
- ❑ **Syntaxe de base :**

```
public class Utilitaire {  
    public static <T> void afficher(T element) {  
        System.out.println("Element : " + element);  
    }  
}
```

- ❑ **Exemple : Appeler une Méthode Générique**

```
public class TestGenerique {  
    public static void main(String[] args) {  
        Utilitaire.afficher(123);           // Affiche 123  
        Utilitaire.afficher("Bonjour");     // Affiche Bonjour  
        Utilitaire.afficher(3.14);          // Affiche 3.14  
    }  
}
```


Création de méthodes Génériques

❑ Exemple : Retourner un Type Générique

```
public class Utilitaire {  
    public static <T> T retournerElement(T element) {  
        return element;  
    }  
}  
  
public class TestGenerique {  
    public static void main(String[] args) {  
        int nombre = Utilitaire.retournerElement(42);  
        System.out.println("Nombre : " + nombre);  
  
        String texte = Utilitaire.retournerElement("Hello");  
        System.out.println("Texte : " + texte);  
    }  
}
```

Contraintes sur les Types (Bounded Types)

- ❑ Il est possible de restreindre les types qui peuvent être utilisés avec une classe ou une méthode générique en utilisant des bornes.
- ❑ Cela permet de garantir que le type passé respecte certaines propriétés, comme hériter d'une classe particulière ou implémenter une interface spécifique.

Contraintes sur les Types (Bounded Types)

- ❑ Les bornes sont définies à l'aide de **extends** (pour une classe ou une interface) ou **super** (dans certains cas avec des wildcards).
- ❑ **Syntaxe de base :**

```
public class ClasseAvecBorne<T extends Number> {  
    private T element;  
  
    public ClasseAvecBorne(T element) {  
        this.element = element;  
    }  
  
    public T getElement() {  
        return element;  
    }  
}
```

Number est une classe abstraite du package **java.lang**, utilisée comme superclasse des classes représentant des valeurs numériques : **Integer**, **Double**, **Float**, **Long**, **Short**, et **Byte**.

Contraintes sur les Types (Bounded Types)

- ❑ **Exemple** : Classe Générique Limitée aux Sous-Types de Number

```
public class TestGenerique {  
    public static void main(String[] args) {  
        ClasseAvecBorne<Integer> entier = new ClasseAvecBorne<>(10);  
        System.out.println("Valeur : " + entier.getElement());  
  
        ClasseAvecBorne<Double> decimal = new ClasseAvecBorne<>(5.75);  
        System.out.println("Valeur : " + decimal.getElement());  
  
        // ClasseAvecBorne<String> erreur = new ClasseAvecBorne<>("Test");  
        // Erreur de compilation  
    }  
}
```

Contraintes sur les Types (Bounded Types)

❑ Exemple : Méthode Générique avec Borne

```
public class Utilitaire {  
    // Méthode générique avec une borne sur Number  
    public static <T extends Number> double calculerMoyenne(T a, T b) {  
        return (a.doubleValue() + b.doubleValue()) / 2.0;  
    }  
}  
  
public class TestGenerique {  
    public static void main(String[] args) {  
        // Moyenne de deux entiers  
        System.out.println("Moyenne : " + Utilitaire.calculerMoyenne(10, 20));  
  
        // Moyenne de deux nombres décimaux  
        System.out.println("Moyenne : " + Utilitaire.calculerMoyenne(5.5, 8.5));  
  
        // Moyenne de deux types différents (Integer et Double)  
        System.out.println("Moyenne : " + Utilitaire.calculerMoyenne(3, 4.5));  
    }  
}
```

Contraintes sur les Types (Bounded Types)

❑ Exemple : Méthode Générique avec Borne

```
public class Utilitaire {  
    public static <T extends Comparable<T>> T max(T a, T b) {  
        return (a.compareTo(b) > 0) ? a : b;  
    }  
}  
  
public class TestGenerique {  
    public static void main(String[] args) {  
        System.out.println("Maximum : " + Utilitaire.max(5, 10)); // Affiche 10  
        System.out.println("Maximum : " + Utilitaire.max("A", "Z")); // Affiche Z  
    }  
}
```

Contraintes sur les Types (Bounded Types)

❑ Exemple : Méthode Générique avec Borne

```
import java.util.Collection;

public class Utilitaire {
    // Méthode générique avec une borne sur Collection
    public static <T extends Collection<?>> int calculerTailleTotale(T collection) {
        return collection.size();
    }
}

public class TestGenerique {
    public static void main(String[] args) {
        // Liste d'entiers
        Collection<Integer> liste = java.util.List.of(1, 2, 3, 4, 5);
        System.out.println("Taille de la liste : " + Utilitaire.calculerTailleTotale(liste))

        // Ensemble de chaînes
        Collection<String> ensemble = java.util.Set.of("A", "B", "C");
        System.out.println("Taille de l'ensemble : " + Utilitaire.calculerTailleTotale(ensem
```

Contraintes sur les Types (Bounded Types)

- ❑ **Exemple supplémentaire** : Méthode avec plusieurs bornes

```
public class Utilitaire {
    public static <T extends Number & Comparable<T>> T min(T a, T b) {
        return (a.compareTo(b) < 0) ? a : b;
    }
}

public class TestGenerique {
    public static void main(String[] args) {
        System.out.println("Minimum : " + Utilitaire.min(3.5, 2.7)); // Affiche 2.7
        System.out.println("Minimum : " + Utilitaire.min(10, 20));    // Affiche 10
    }
}
```


Contraintes sur les Types (Bounded Types)

□ Cas d'utilisation :

- 1. Traitement de nombres** : Par exemple, une classe ou méthode générique pour des calculs arithmétiques ou des comparaisons.
- 2. Collections personnalisées** : Création de structures de données qui fonctionnent uniquement avec des types spécifiques.
- 3. Validation et filtrage** : Garantir qu'un type respecte certaines contraintes métier.

Avantages de la Généricité

- **Sécurité des types à la compilation** : Permet de détecter les erreurs de type avant l'exécution.
- **Réutilisation du code** : Les classes et méthodes génériques peuvent être réutilisées avec différents types de données.
- **Code plus lisible** : La généricité élimine le besoin de conversions explicites (cast).

Conclusion

- ❑ La généricité est un outil puissant en Java pour écrire des classes et méthodes flexibles tout en maintenant une forte sécurité des types.
- ❑ Elle permet de rendre le code plus réutilisable, propre et sécurisé.